CSIS 1230: Programming for Everyone II

Course Outline - Spring 2025

Prerequisites: CSIS 1210 (Introduction to Python Programming)
Meeting Times: Monday/Wednesday/Friday, 9:15 am – 10:20 am
Venue: Carlson Tower, 601
Programming Language: Python 3.x
Instructor: Dr. Clement Essien
Office Hours: Monday 10:30 am – 1pm, Wednesday 10:30 am – 2 pm
Contact: cessien@northpark.edu

Course Description

This course builds upon foundational programming concepts to introduce object-oriented programming, web application development, and database integration. Students will learn to design and implement full-stack applications using Python, Flask, and PostgreSQL while developing proficiency in accessing data from the internet via API consumption, data validation, and software-engineering best practices including version control, testing, and documentation.

Course Objectives

Upon successful completion of this course, students will be able to:

1. Design and implement object-oriented solutions using classes, inheritance, and polymorphism
2. Consume and parse data from web APIs in JSON and XML formats
3. Develop web applications using the Flask framework with full CRUD functionality
4. Design and query relational databases using SQL and PostgreSQL
5. Integrate Python applications with databases using psycopg2
6. Apply testing methodologies using pytest framework
7. Utilize version control systems (Git/GitHub) for collaborative development
8. Create comprehensive technical documentation for software projects

---

NORTH PARK UNIVERSITY
CHICAGO

Required Materials

Software Requirements

- Python 3.8 or higher
- PostgreSQL database
- Git version control system
- Integrated Development Environment (VS Code recommended)

Online Resources

- Course materials available via Canvas Learning Management System
- GitHub for version control and project submission
- PostgreSQL documentation
- Flask documentation

Module 1: Object Oriented Programming

Week 1

M - Course introduction, syllabus overview, classes and objects, attributes and methods, understanding self
W - `__init__` method, `__str__` vs `__repr__`, instance vs class attributes, stateful classes, static vs class methods and when not to use self
F - Lab: Building a stateful class

Week 2

M - **Martin Luther King Jr. Day (University Holiday)**
W - Encapsulation: public and private attributes, getters and setters, properties
F - Lab: Creating your project repository (Project discussion)

Week 3

M - Inheritance hierarchies, method overriding
W - Polymorphism, Abstract Base Classes (ABC) (Project discussion)
F - Lab: Implementing inheritance and abstract classes

NORTH PARK UNIVERSITY
CHICAGO

Module 2: APIs & Handling Structured Data

Week 4

M - Introduction to web data and web APIs: HTTP methods, authentication, status codes
W - requests library: GET/POST methods, parameters, headers
F - Lab: Fetch API data → Parse JSON → Create OOP objects

Week 5

M - XML parsing (ElementTree), HTML parsing, data conversion strategies
W - Data validation: type checking, sanitization, error handling; XML vs JSON parsing
F - Lab: Building an XML parser

Week 6

M - API error resilience: retries, exponential backoff, rate limits
W - Structured error handling with custom exceptions
F - Lab: **Milestone 1** - HTTP request plus server response on GitHub repository

---

Module 3: Web Application Development with Flask

Week 7

M - Flask setup: routes, views, request/response cycle
W - Virtual environment project setup and dependencies installation
F - Lab: Display response data from server on web pages

Week 8

M - Web UI design and templates
W - Full CRUD operations: Create/Read/Update/Delete patterns
F - Lab: **Milestone 2** - CRUD pattern implementation on the server

---

SPRING BREAK - March 9-March 15 (No Classes)

NORTH PARK UNIVERSITY
CHICAGO

Module 4: Databases & SQL

Week 9

M - Database design: tables, relationships, normalization (PostgreSQL introduction)
W - SQL CRUD operations: INSERT/SELECT/UPDATE/DELETE
F - Lab: Create project database schema in PostgreSQL

Week 10

M - SQL aggregate functions: COUNT, SUM, AVG, GROUP BY
W - Advanced SQL: JOIN operations (INNER, LEFT, RIGHT, FULL)
F - Lab: Complex queries on project data

Week 11

M - psycopg2: Integrating Python with PostgreSQL
W - ORM patterns: Class ↔ Table mapping strategies
F - **Good Friday (University Holiday)**

Module 5: Testing & Quality Assurance

Week 12

M - **Easter Monday (University Holiday)**
W - Debugging techniques: breakpoints, stack traces, logging; fail-safe code with try/except
F - pytest framework: fixtures, parametrization, mocking

Module 6: Version Control, Documentation, and Final Project Integration

Week 13

M - Git workflows: branching, merging, GitHub collaboration
W - Documentation best practices: docstrings, Sphinx, README files, user guides
F - Lab: **Milestone 3** - Storage layer complete (API data → PostgreSQL)

Week 14

M - In-class development workshop: Focus on coding and integrating all project components with instructor guidance
W - Testing and polish: Final testing and project refinement with instructor support
F - Lab: **Milestone 4** - Fully integrated project with comprehensive README.md

Week 15

M - Final project presentations: Part 1
W - Final project presentations: Part 2
F - Final project presentations: Part 3

Week 16

M - Final project presentations: Part 4
W - Final project presentations: Part 5 with Q&A and peer evaluation
F - Course conclusion and feedback session; course evaluations

---

Final Project

Core Project: Query External API → Database → CRUD Web Service

Students will build a full-stack capstone application that:

- Retrieves live data from an external API (local weather tracker)
- Stores data in a relational PostgreSQL database
- Provides CRUD endpoints to retrieve all records, retrieve by ID, update data by ID, post by ID, and delete records
- Demonstrates integration of all course concepts

Each module incrementally builds and integrates a component of this project.

Assessment Structure

Component	Weight	Description
Classwork	20%	Weekly in-class notebooks and/or Python files submitted to Canvas
Final Project	60%	Full-stack application submitted via GitHub with comprehensive README
Labs / Class Participation	20%	In-class attendance, engagement, pair programming labs, and required readings
TOTAL	**100%**	

Project Submission Requirements

All final projects must be submitted to a GitHub repository with: - Commit history demonstrating progress over time - Comprehensive README.md file including: - Project overview and purpose - Setup and run instructions - Feature descriptions - Sample code if necessary

---

Grading Scale

Letter grades will be assigned according to the following scale:

Grade	Range
A	90-100
B	80-89
C	70-79
D	60-69
F	0-59

Final Project Assessment (60 points total)

Component	Points	Description
Functionality & Completeness	15	Core MVP works fully; stretch goals implemented; essential features (classes, data I/O, web API) present and correct
Code Quality & Style	10	Code is clear, well-named, modular, formatted per PEP8; includes comments and readable structure
Use of Course Concepts	10	Explicit use of OOP and at least one additional topic (API interaction, GUI/webview, database)
Testing & Error Handling	5	Includes unit tests (pytest) and handles exceptions in code execution paths
Documentation & README	10	README provides setup instructions,

Component	Points	Description
		usage guide, project overview; code includes docstrings
Presentation & Demonstration	10	Clear and polished 15-minute presentation; demonstrates features; responds well to questions
TOTAL	**60**	

Project/Classwork Rubric

Category	Excellent	Good	Fair
Functionality / Completion	All features work correctly; at least one stretch goal met; handles edge cases and invalid input gracefully	Most functionality present; shows clear understanding of main topic to solve the problem	Major features missing or only partially implemented
Code Quality & Design	No syntax errors; clean, modular structure; well-organized with helpful comments for readability	Some code structure issues; formatting or commenting is minimal	Poorly structured or undocumented code; lacks readability
Course Topic Integration	Fully integrates multiple course concepts (OOP, Python + SQL integration, API request handling, proper documentation)	Applies some relevant course topics	Only one or few topics applied; limited problem coverage
Version Control Practices	Regular, meaningful Git commits with clear messages; GitHub repository well-maintained	Commits irregular or lack clarity; repository used inconsistently	Minimal or no version control use; few or no commits
Documentation &	README includes setup instructions, features, usage	README is minimal; some	README missing or

Category	Excellent	Good	Fair
README	guide; code includes docstrings	docstrings present	incomplete; no docstrings
Testing	Unit tests cover key functions and edge cases using pytest or unittest	Some tests present	No tests or inadequate coverage
Presentation & Demo	Clear, confident 15-minute demo with walkthrough and Q&A	Demo works but is unpolished or rushed	Demo is broken or incomplete; unable to answer questions

Course Policies

Attendance

Regular attendance is expected and strongly correlated with academic success. All notebooks and Python files for classwork must be uploaded to Canvas by the end of the class session when administered.

Late Work Policy

- **Submissions:** Accepted within 24 hours of deadline for 90% credit. No credit after 24 hours.
- **Examinations:** Missed examinations require official documentation; makeup examinations may differ in format

Academic Integrity

North Park University maintains strict standards of academic honesty. In this course:

- All submitted work must be the student's own
- **The use of AI-generated code or copying code from online sources is strictly prohibited**
- Students must be able to explain and defend all submitted code
- Collaboration on conceptual understanding is encouraged; sharing code solutions is not permitted

Violations will result in: - Automatic failure for the assignment (grade of 0) - Referral to the university Academic Integrity Board for treatment as academic dishonesty in accordance with university policy

Refer to the University policy on integrity and academic honesty.

Accommodations

Students with documented disabilities should contact the Center for Student Engagement to arrange appropriate accommodations. Please notify the instructor within the first two weeks of the semester to ensure timely implementation of accommodation.

Tentative Schedule Note

This syllabus represents the instructor's current plans and objectives. As the semester progresses, circumstances may require modifications to the schedule, topics, or assessment structure. Any such changes will be announced in class and updated on Canvas with reasonable advance notice.